

Recursion:

A problem is broken down into smaller problems to solve a bigger problem.

ex: Sum of N natural numbers

$$\text{Sum}(N) = \underbrace{\text{Sum}(N-1)}_{\text{sub problem}} + N$$

How to write recursive code?

- 1) Assumption: assume that your function works for a smaller input.
- 2) Main logic: Solve bigger problem with sub problems.
- 3) Base case: Populate the answer for the smallest input that you know.

Function call trace:

```
func add (int x, int y){  
    return x+y;  
}
```

```
{  
func mul (int x, int y){  
    return x*y;  
}
```

```
func sub (int x, int y){  
    return x-y;  
}
```

```
{  
func print (int x){  
    SOP(x);  
}
```

```
main() {
```

```
    x = 10;
```

```
    y = 20;
```

```
    print(sub(mul(add(x, y), 70), 40));
```

```
}
```

X	add x = 10 y = 20 return 30
X	mul x = 30 y = 70 return 2100
X	sub x = 2100 y = 40 return 2060
X	print x = 2060
X	main x = 10 y = 20

Function call stack

2060

O/P

Question: Given N , find its factorial.

$$\text{fact}(N) = N! = N * (N-1) * (N-2) * \dots * 1$$

ex: $5! = 5 * 4 * 3 * 2 * 1 = 120$

$$\text{fact}(N) = N * \underbrace{\text{fact}(N-1)}_{\text{sub problem}}$$

→ Recurrence relation
→ Step 1

$N = 5$

```
func factorial (int N) {  
    // Base case  
    if (N == 1) return 1;  
    // Step 2  
    return N * factorial(N-1);  
}
```

```
main () {  
    print (factorial(5));  
}
```

1	factorial $N = 1$
2	factorial $N = 2$
6	factorial $N = 3$
24	factorial $N = 4$
120	factorial $N = 5$
	main

Question: Print N numbers in increasing order
from 1

Recurrence relation

$$Inc(N) = \underbrace{Inc(N-1)}_{\text{sub problem}} + \text{print}(N)$$

```
func inc(int N) {  
    // Step 3  
    if (N == 1) {  
        print(1);  
        return; }  
    // Step 2  
    inc(N-1)  
    print(N)  
}
```



O/P

1 2 3 4 5

$$T.C = O(1 * N) = O(N)$$

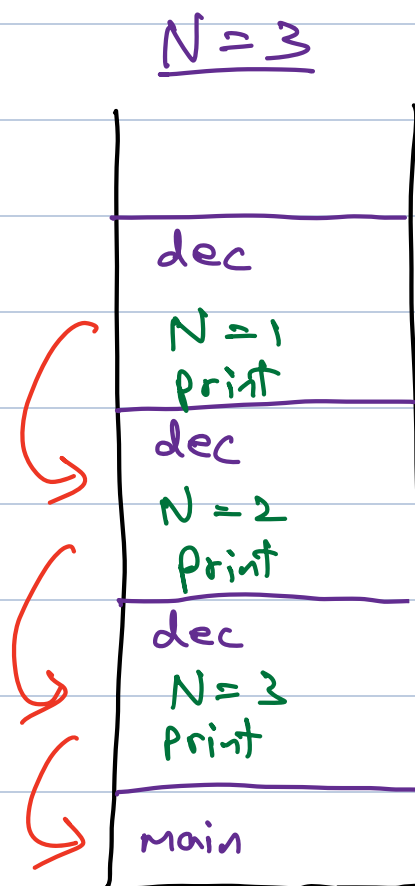
$$S.C = O(N + 1) = O(N)$$

Question: Print N numbers in decreasing order.
till 1

Recurrence relation

$$\text{dec}(N) = \text{print}(N) + \text{dec}(N-1)$$

```
func dec(int N) {  
    // Base case  
    if (N == 1) {  
        print(1);  
        return;  
    }  
    // Step 2  
    print(N)  
    dec(N-1)  
}
```



O/P

3 2 1

$$\begin{aligned} T.C &= O(1 * N) \\ &= O(N) \end{aligned}$$

$$\begin{aligned} S.C &= O(N + 1) \\ &= O(N) \end{aligned}$$

Time and Space Complexity of Recursive Code

Time Complexity = $O(\# \text{ function calls} * \text{TC of each func})$

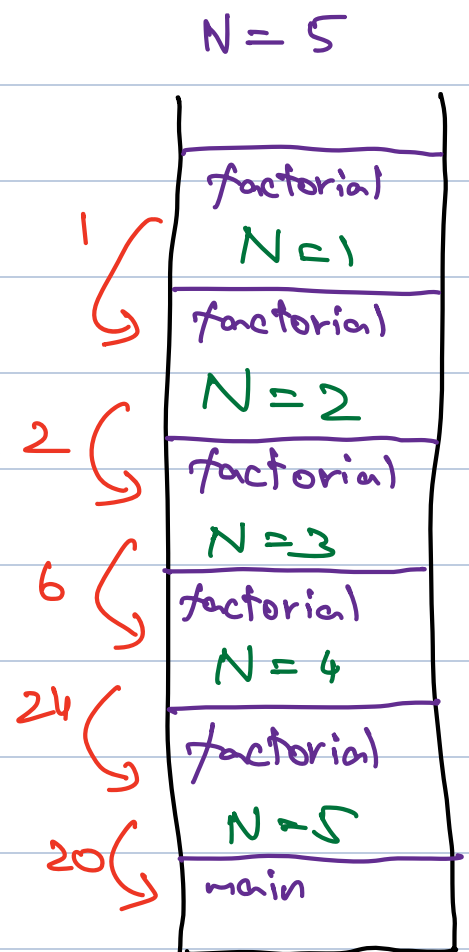
Space Complexity = $O(\text{Max depth of recursive stack} + \text{Space per function call})$

```
func factorial (int N) {  
    // Base case  
    if (N == 1) return 1;  
    // Step 2  
    return N * factorial(N-1);  
}
```

T.C of each = $O(1)$

funcⁿ call

of calls = N



$$T.C = O(1 * N) = N$$

$$S.C = O(N + 1) \\ = O(N)$$

Question: Given N , print the N th number in the Fibonacci Series.

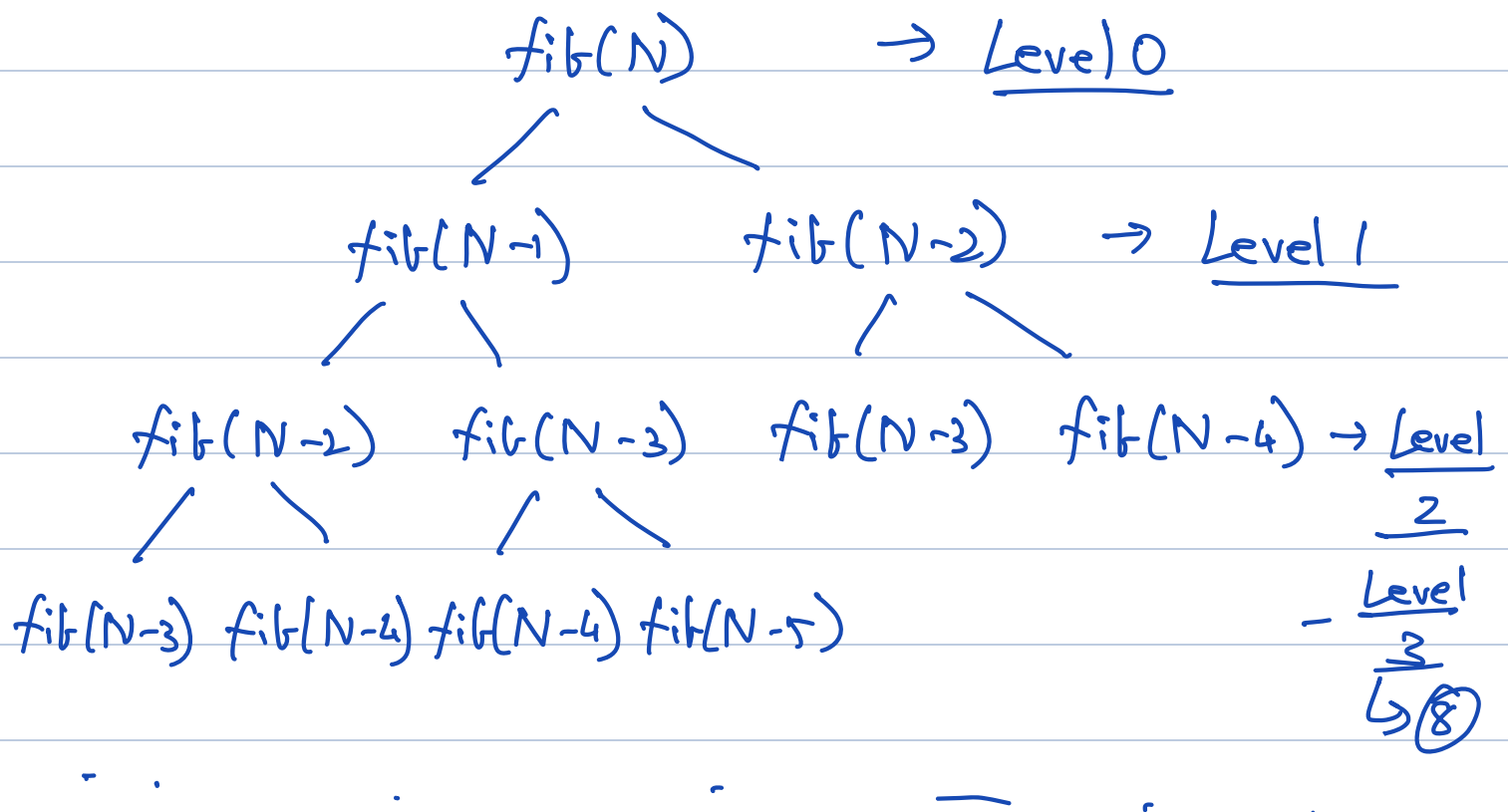
0 1 2 3 4 5 6 7

Fibonacci = 0 1 1 2 3 5 8 13 . . .
Series

Recurrence relation

$$\text{fib}(N) = \text{fib}(N-1) + \text{fib}(N-2)$$

```
func fib(int N) {  
    // Base case  
    if (N == 0) return 0;  
    if (N == 1) return 1;  
    // Recurrence relation  
    return fib(N-1) + fib(N-2);  
}
```



Nth level = 2^N funcⁿ calls

Total levels = N

Total funcⁿ calls = Sum of all levels.

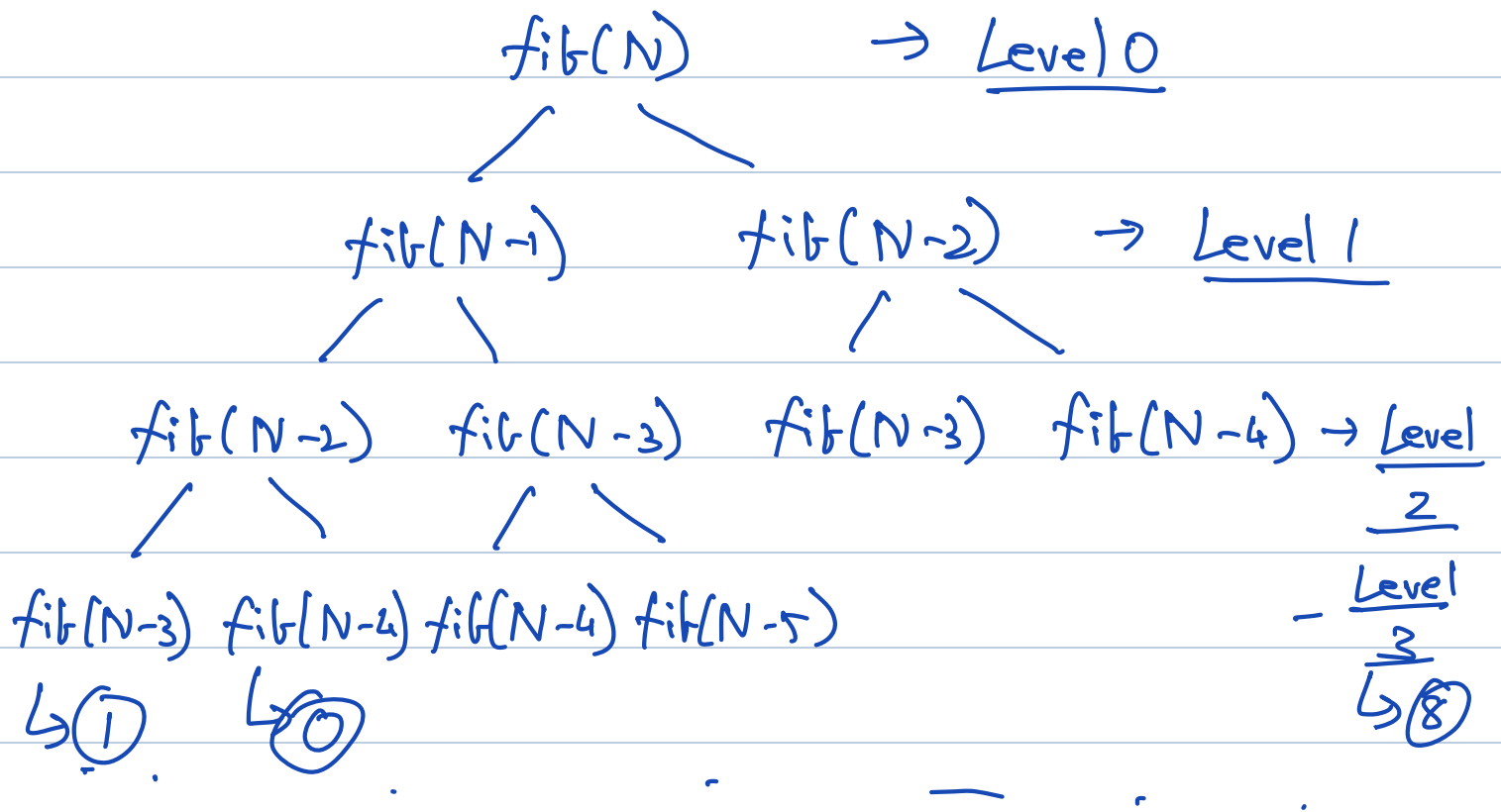
$$= 1 + 2 + 4 + \dots + 2^N$$

$$a=1 \quad r=2$$

$$\text{Sum} = \frac{a(r^N - 1)}{r - 1}$$

$$\text{Total func}^n \text{ calls} = \frac{1(2^N - 1)}{2 - 1} = 2^N - 1$$

$$T.C = O(1 * (2^N - 1)) = O(2^N)$$



Total levels = N

Depth = Levels

$$S.C = O(N+1) \approx O(N)$$

$$\text{fib}(2) \rightarrow 1+0 = 1$$

$$\text{fib}(1) \rightarrow 1 \quad \text{fib}(0) \rightarrow 0$$

```
func expensive (N) {  
  arr[N]  
  expensive (N-1)  
}
```

$N = N$

```
func expensive (N) {  
  arr[N]  
  expensive (N-1)  
}
```

$N = N - 1$

~~func expensive (N) {~~

```
func expensive (N) {  
  arr[N]  
  expensive (N-1)  
}
```

$N = N - 2$

~~func expensive (N) {~~

```
func expensive (N) {  
  arr[N]  
  expensive (N-1)  
}
```

$N = 1$

$\rightarrow$ ~~func expensive (N) {~~